

For example, tools like GitHub Copilot do not just suggest snippets anymore, they often provide complete solutions, debug issues, or even generate test cases. In many ways, AI has become a co-programmer rather than a passive assistant.

From automation to intelligent automation

For years, the industry has been focused on automation, finding ways to streamline repetitive tasks and improve efficiency. But with AI in the mix, we are talking about *intelligent* automation.

Platforms like Blue Prism, for instance, started out with robotic process automation. Now, they are evolving into intelligent automation, where processes are not just automated but are also adaptive, data-driven, and context-aware.

This is exactly what is happening in software development. Developers are no longer just code writers, they are co-creators with AI, embedding intelligence into products whenever it adds real value for end users and businesses.

But here is the critical point — not every software product needs AI. The main questions organisations must ask are:

- Does AI improve productivity for developers?
- Does AI deliver measurable value to end users?
- Does it contribute to revenue growth?

If the answer is ‘no’, then adding AI is just noise. Business leaders are rightly prioritising usefulness and ROI over hype.

Pain points of traditional development and how AI helps

While I hesitate to call it traditional development, it is clear that we are seeing a big shift in how things are being done today. Let me highlight some key pain points that AI is addressing.

Manual debugging: Debugging has always been time-consuming. Tools like GitHub Copilot now highlight errors in real time, suggesting fixes instantly.

Repetitive coding: Starting projects often meant rewriting boilerplate code. Now, a single line or keyword can trigger AI to suggest entire code blocks, saving significant time.

Testing: Unit testing used to be a burden. Today, AI generates test cases and testing frameworks, reducing repetitive effort.

Onboarding new developers: In large organisations, locating project files across platforms like SharePoint, GitHub, or ServiceNow can be overwhelming for new hires. AI-powered chatbots or plugins can centralise knowledge, making onboarding faster and smoother.

Skill gaps: This is something I experienced firsthand. I once joined a project built in Golang, even though my core expertise was in Java. Initially, I had no idea where to start. GitHub Copilot became my guide—explaining the workflow, clarifying syntax and even breaking down code line by line. It not only got the work done but helped me learn Golang in the process.

GitHub Copilot: An AI pair programmer

Let’s talk specifically about GitHub Copilot, which I have personally used extensively.

We do not call it an AI programmer. Instead, it is an AI pair programmer, a collaborator that supports but does not replace human creativity. Here are some ways it has transformed my work.

Fixing code in-line: Copilot highlights errors (with its familiar yellow star icon) and suggests corrections instantly.

Understanding new code: Perfect for working with unfamiliar languages or legacy projects.

Unit testing and documentation: Writing documentation has always been a pain point. Copilot auto-generates drafts, saving hours of manual effort.

PR summaries and repo interaction: It can generate pull request summaries and even “talk” to repositories.

Error handling: When errors show up in the terminal, Copilot immediately suggests possible fixes.

One note of caution is to always ensure that you are not sharing sensitive code or data with these tools. Many organisations are rightfully concerned about data security in AI-assisted workflows.

Final thoughts

AI is no longer just an accessory in software development; it is becoming a collaborator. From debugging and testing to onboarding and documentation, tools like GitHub Copilot are easing pain points that developers have lived with for years. But while the benefits are clear, it’s important that AI should add genuine value. Whether it is improving developer productivity or enhancing the end-user experience, the goal must always be impact and not hype.

As developers, we should embrace AI as a thought partner, a co-pilot, and a catalyst for innovation, while also being mindful of its limitations and risks. The journey from ‘writing code’ to ‘intelligent collaboration’ has just begun, and I am excited to see where it takes us. **END** 🐧

*The article is based on the talk titled ‘Transforming Software Development with AI’ given by **Harsh Sharma (Shell)** at AIDevCon 2025 in Bengaluru. It has been transcribed and developed by **Vidushi Saxena**, a journalist at the EFY Group.*